Introduction to Diffusion Models

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Foundations of Generative Deep Learning
2. Evaluation Metrics
3. The Diffusion Phenomena
4. Accelerating the Generations(DDIM)
5. Guidance and Controlled generation
6. The Latent Diffusion Models

Foundations of Generative Deep Learning

The Generative Landscape

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.
- **Asks:** What makes an image look like a cat?

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.
- **Asks:** What makes an image look like a cat?
- Examples: Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs).

Discriminative vs. Generative models

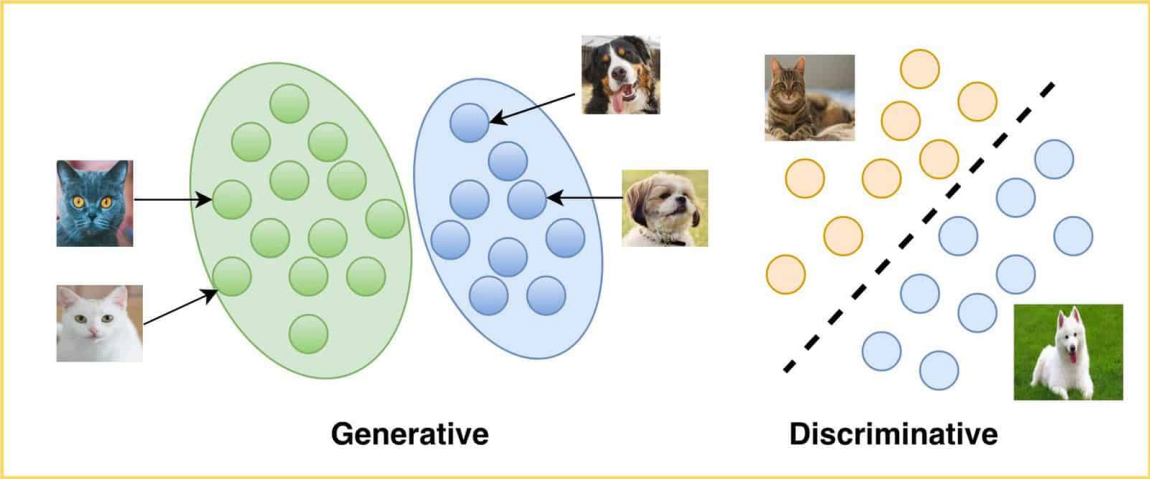


Figure: Different Learning patterns of Discriminative and Generative models.(source: learnopencv.com

Taxonomy of Generative Models

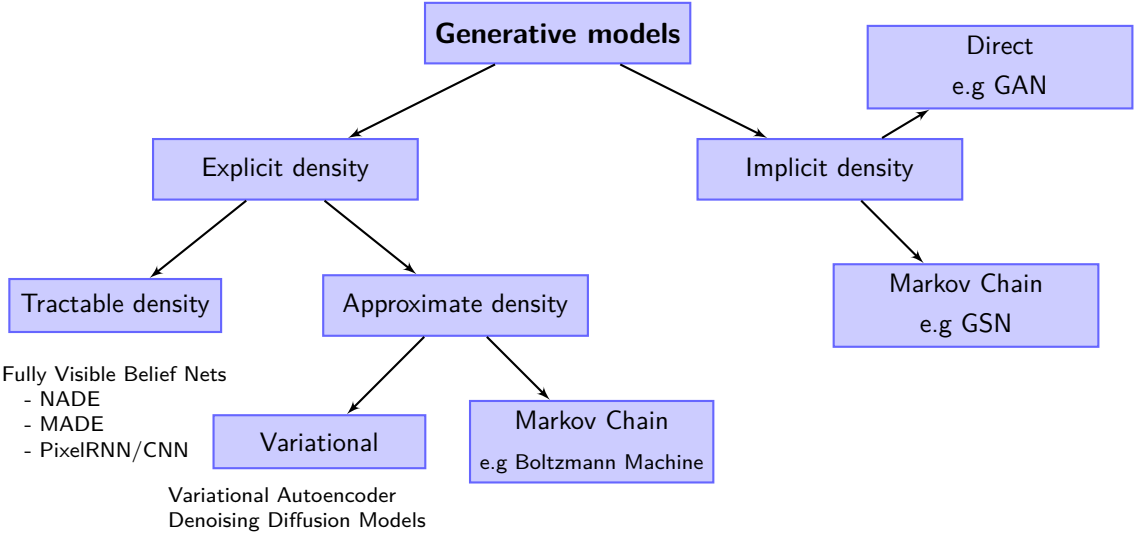


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

VAE vs. GAN: Two Different Philosophies

How do they approach generation?

Variational Autoencoders (VAE)

The "Compressor"

- **Goal:** Learn the underlying distribution of the data by compressing it.

VAE vs. GAN: Two Different Philosophies

How do they approach generation?

Variational Autoencoders (VAE)

The "Compressor"

- **Goal:** Learn the underlying distribution of the data by compressing it.
- **Analogy:** An artist looks at a face, memorizes the key features (eye color, shape), and tries to redraw it from memory.

VAE vs. GAN: Two Different Philosophies

How do they approach generation?

Variational Autoencoders (VAE)

The "Compressor"

- **Goal:** Learn the underlying distribution of the data by compressing it.
- **Analogy:** An artist looks at a face, memorizes the key features (eye color, shape), and tries to redraw it from memory.
- **Type:** Explicit (Approximate) Density. We try to maximize the likelihood of the data.

Generative Adversarial Networks (GAN)

The "Imitator"

- **Goal:** Fool a classifier into thinking generated images are real.

VAE vs. GAN: Two Different Philosophies

How do they approach generation?

Variational Autoencoders (VAE)

The "Compressor"

- **Goal:** Learn the underlying distribution of the data by compressing it.
- **Analogy:** An artist looks at a face, memorizes the key features (eye color, shape), and tries to redraw it from memory.
- **Type:** Explicit (Approximate) Density. We try to maximize the likelihood of the data.

Generative Adversarial Networks (GAN)

The "Imitator"

- **Goal:** Fool a classifier into thinking generated images are real.
- **Analogy:** An art forger tries to paint a fake Picasso. An art critic tries to spot the fake. They both get better by competing.

VAE vs. GAN: Two Different Philosophies

How do they approach generation?

Variational Autoencoders (VAE)

The "Compressor"

- **Goal:** Learn the underlying distribution of the data by compressing it.
- **Analogy:** An artist looks at a face, memorizes the key features (eye color, shape), and tries to redraw it from memory.
- **Type:** Explicit (Approximate) Density. We try to maximize the likelihood of the data.

Generative Adversarial Networks (GAN)

The "Imitator"

- **Goal:** Fool a classifier into thinking generated images are real.
- **Analogy:** An art forger tries to paint a fake Picasso. An art critic tries to spot the fake. They both get better by competing.
- **Type:** Implicit Density. We don't model probability; we just learn to sample.

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \rightarrow$ Encoder $\rightarrow$ Latent Dist $q(z|x) \rightarrow$ Sample $z \rightarrow$ Decoder $\rightarrow$ Output $\hat{x}$.

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \to$ Encoder $\to$ Latent Dist $q(z|x) \to$ Sample $z \to$ Decoder $\to$ Output $\hat{x}$.
- **Objective:** maximize ELBO.

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \rightarrow$ Encoder $\rightarrow$ Latent Dist $q(z|x) \rightarrow$ Sample $z \rightarrow$ Decoder $\rightarrow$ Output $\hat{x}$.
- **Objective:** maximize ELBO.
- **Loss Function:**

$$\mathcal{L} = \underbrace{||x - \hat{x}||^2}_{\text{Reconstruction}} + \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization}}$$

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \to$ Encoder $\to$ Latent Dist $q(z|x) \to$ Sample $z \to$ Decoder $\to$ Output $\hat{x}$.
- **Objective:** maximize ELBO.
- **Loss Function:**

$$\mathcal{L} = \underbrace{||x - \hat{x}||^2}_{\text{Reconstruction}} + \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization}}$$

- *Intuition:* "Make it look like the input, but keep the latent space organized."

GAN: Two-Player Game

- **Process:**
  - **Generator (G):** Noise $z \to x_{fake}$.
  - **Discriminator (D):** Classifies x_{real} vs x_{fake}.

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \to$ Encoder $\to$ Latent Dist $q(z|x) \to$ Sample $z \to$ Decoder $\to$ Output $\hat{x}$.
- **Objective:** maximize ELBO.
- **Loss Function:**

$$\mathcal{L} = \underbrace{||x - \hat{x}||^2}_{\text{Reconstruction}} + \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization}}$$

- *Intuition:* "Make it look like the input, but keep the latent space organized."

GAN: Two-Player Game

- **Process:**
  - **Generator (G):** Noise $z \to x_{fake}$.
  - **Discriminator (D):** Classifies x_{real} vs x_{fake}.
- **Objective:** Minimax Game.

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \to$ Encoder $\to$ Latent Dist $q(z|x) \to$ Sample $z \to$ Decoder $\to$ Output $\hat{x}$.
- **Objective:** maximize ELBO.
- **Loss Function:**

$$\mathcal{L} = \underbrace{||x - \hat{x}||^2}_{\text{Reconstruction}} + \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization}}$$

- *Intuition:* "Make it look like the input, but keep the latent space organized."

GAN: Two-Player Game

- **Process:**
  - **Generator (G):** Noise $z \to x_{fake}$.
  - **Discriminator (D):** Classifies x_{real} vs x_{fake}.
- **Objective:** Minimax Game.
- **Loss Function:**

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

VAE vs. GAN: Under the Hood

Architecture and Objectives

VAE: Encoder-Decoder

- **Process:** Input $x \to$ Encoder $\to$ Latent Dist $q(z|x) \to$ Sample $z \to$ Decoder $\to$ Output $\hat{x}$.
- **Objective:** maximize ELBO.
- **Loss Function:**

$$\mathcal{L} = \underbrace{||x - \hat{x}||^2}_{\text{Reconstruction}} + \underbrace{D_{KL}(q(z|x)||p(z))}_{\text{Regularization}}$$

- *Intuition:* "Make it look like the input, but keep the latent space organized."

GAN: Two-Player Game

- **Process:**
  - **Generator (G):** Noise $z \to x_{fake}$.
  - **Discriminator (D):** Classifies x_{real} vs x_{fake}.
- **Objective:** Minimax Game.
- **Loss Function:**

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

- *Intuition:* "D tries to catch fakes; G tries to fool D."

VAE vs. GAN: Under the Hood

Architecture and Objectives

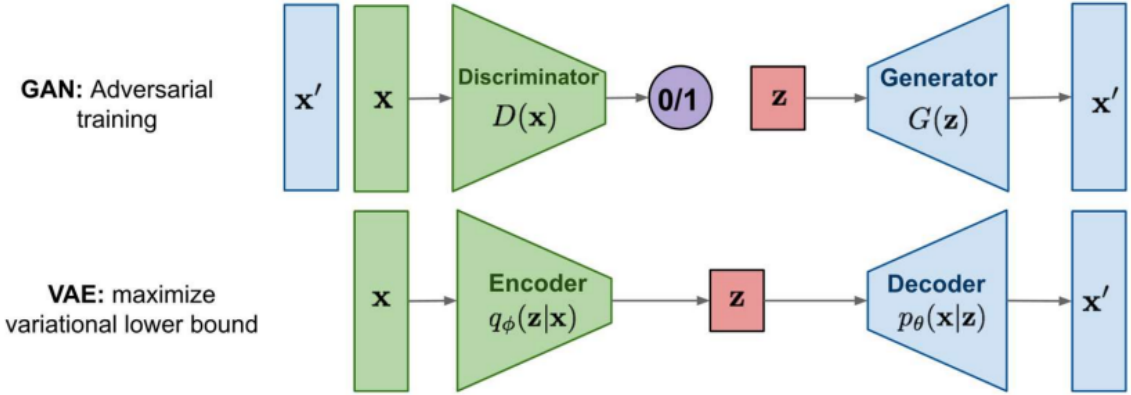


Figure: Architetural difference between VAE and GANs

The Generative Trilemma: Why Diffusion?

Comparing the Results

	VAE	**GAN**
Image Quality	Low (Blurry). The MSE loss tends to "average" out fine details.	High (Sharp). The Discriminator forces realism.
Diversity	High. Good coverage of the data distribution (Mass covering).	Low (Mode Collapse). The generator finds one image that fools D and keeps repeating it.
Training	Stable. Easy to train, converges well.	Unstable. Hard to balance the two networks (oscillations).

The Generative Trilemma: Why Diffusion?

Comparing the Results

	VAE	**GAN**
Image Quality	Low (Blurry). The MSE loss tends to "average" out fine details.	High (Sharp). The Discriminator forces realism.
Diversity	High. Good coverage of the data distribution (Mass covering).	Low (Mode Collapse). The generator finds one image that fools D and keeps repeating it.
Training	Stable. Easy to train, converges well.	Unstable. Hard to balance the two networks (oscillations).

The Gap for Diffusion

Question: Can we get the *Stability* and *Diversity* of VAEs with the *Sharpness* of GANs?
Answer: Yes, by iteratively refining the noise (Diffusion).

Evaluation Metrics for Generative models

Defining FID score, IS and CLIP score

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment
- Trade-offs between diversity and quality

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment
- Trade-offs between diversity and quality

Key Questions:

- Is the output realistic?
- Is there sufficient diversity?
- Does it match the training distribution?

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment
- Trade-offs between diversity and quality

Key Questions:

- Is the output realistic?
- Is there sufficient diversity?
- Does it match the training distribution?

Common Metrics

- **FID Score**

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment
- Trade-offs between diversity and quality

Key Questions:

- Is the output realistic?
- Is there sufficient diversity?
- Does it match the training distribution?

Common Metrics

- **FID Score**
- **Inception Score**

Evaluation Metrics for Generative Models

How do we measure success?

The Challenge:

- Evaluating generative models is like evaluating art
- No single "correct" output
- Subjective quality assessment
- Trade-offs between diversity and quality

Key Questions:

- Is the output realistic?
- Is there sufficient diversity?
- Does it match the training distribution?

Common Metrics

- **FID Score**
- **Inception Score**
- **CLIP Score**

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

- Compare statistics of real vs generated images

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

- Compare statistics of real vs generated images
- Uses features from Inception-v3 network

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

- Compare statistics of real vs generated images
- Uses features from Inception-v3 network
- Measures distance between two Gaussian distributions

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

- Compare statistics of real vs generated images
- Uses features from Inception-v3 network
- Measures distance between two Gaussian distributions

Mathematical Formulation:

$$\text{FID} = ||\mu_r - \mu_g||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

where:

- μ_r, Σ_r: mean and covariance of real images
- μ_g, Σ_g: mean and covariance of generated images

FID Score (Frechet Inception Distance)

Measuring distribution similarity

Key Idea:

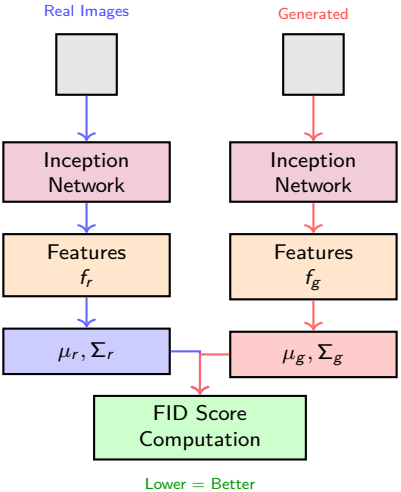
- Compare statistics of real vs generated images
- Uses features from Inception-v3 network
- Measures distance between two Gaussian distributions

Mathematical Formulation:

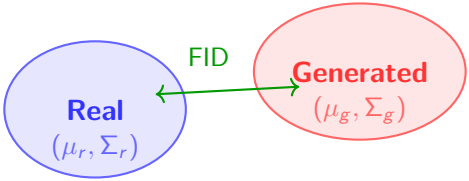
$$\text{FID} = ||\mu_r - \mu_g||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

where:

- μ_r, Σ_r: mean and covariance of real images
- μ_g, Σ_g: mean and covariance of generated images

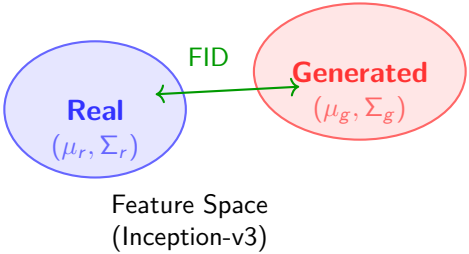


FID Score



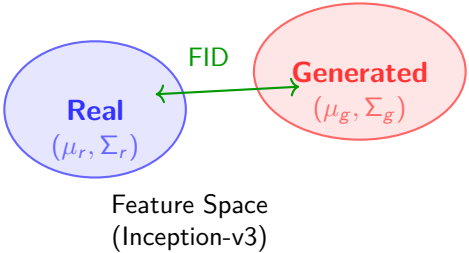
Feature Space
(Inception-v3)

FID Score



Interpretation

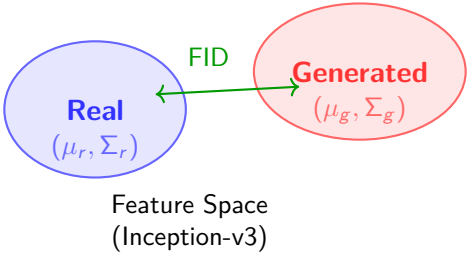
FID Score



Interpretation

- **Lower is better** (0 = identical distributions)

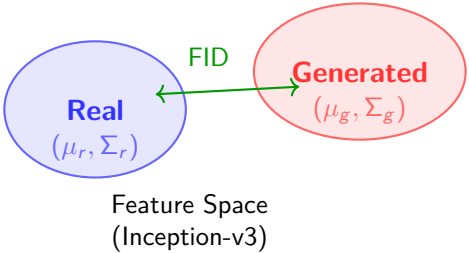
FID Score



Interpretation

- **Lower is better** (0 = identical distributions)
- Sensitive to both quality and diversity

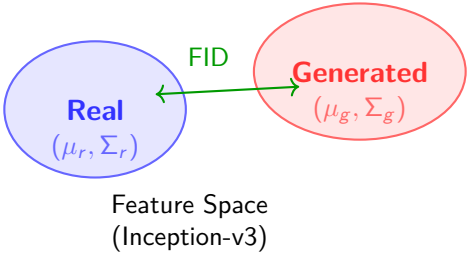
FID Score



Interpretation

- **Lower is better** (0 = identical distributions)
- Sensitive to both quality and diversity
- Most widely used metric in practice

FID Score



Interpretation

- **Lower is better** ($0 =$ identical distributions)
- Sensitive to both quality and diversity
- Most widely used metric in practice
- Typical good FID: < 10 (domain-dependent)

Inception Score (IS)

Measuring quality and diversity

Two Desirable Properties:

1. **Quality**: Generated images should be clearly classifiable
   - Sharp, realistic features
   - Low entropy $p(y|x)$

Inception Score (IS)

Measuring quality and diversity

Two Desirable Properties:

1. **Quality**: Generated images should be clearly classifiable
   - Sharp, realistic features
   - Low entropy $p(y|x)$
2. **Diversity**: Model should generate varied classes
   - Diverse outputs
   - High entropy $p(y)$

Inception Score (IS)

Measuring quality and diversity

Two Desirable Properties:

1. **Quality**: Generated images should be clearly classifiable
   - Sharp, realistic features
   - Low entropy $p(y|x)$
2. **Diversity**: Model should generate varied classes
   - Diverse outputs
   - High entropy $p(y)$

Formula:

$$\text{IS} = \exp(\mathbb{E}_x[\text{KL}(p(y|x)||p(y))])$$

Inception Score (IS)

Measuring quality and diversity

Two Desirable Properties:

1. **Quality**: Generated images should be clearly classifiable
   - Sharp, realistic features
   - Low entropy $p(y|x)$
2. **Diversity**: Model should generate varied classes
   - Diverse outputs
   - High entropy $p(y)$

Formula:

$$\text{IS} = \exp(\mathbb{E}_x[\text{KL}(p(y|x)||p(y))])$$

Characteristics

- **Higher is better** (typical range: 2-10)
- Uses pre-trained Inception-v3 classifier
- Limitations: Biased toward ImageNet classes, doesn't compare to real data

Inception Score (IS)

Step-by-Step Calculation

The Computational Pipeline:

1. **Generate & Classify**: Generate N images (x_i). Pass each through a pre-trained InceptionV3 network to get the conditional label distribution:

$$p(y|x_i) \quad \text{(probability distribution via softmax)}$$

$\rightarrow$ *If image is high quality, this vector is "sharp" (low entropy).*

Inception Score (IS)

Step-by-Step Calculation

The Computational Pipeline:

1. **Generate & Classify**: Generate N images (x_i). Pass each through a pre-trained InceptionV3 network to get the conditional label distribution:

$$p(y|x_i) \quad \text{(probability distribution via softmax)}$$

$\rightarrow$ *If image is high quality, this vector is "sharp" (low entropy).*

2. **Compute Marginal Distribution**: Calculate the average label distribution across all generated images:

$$\hat{p}(y) = \frac{1}{N} \sum_{i=1}^{N} p(y|x_i)$$

$\rightarrow$ *If images are diverse, this vector is "flat" (high entropy).*

Inception Score (IS)

Step by Step Calculation

3. **Measure Divergence (KL)**: For each image, compare its specific label distribution to the average distribution:

$$d_i = \mathrm{KL}(p(y|x_i)||\hat{p}(y))$$

$\rightarrow$ *We want the specific image to be distinct from the average soup.*

Inception Score (IS)

Step by Step Calculation

3. **Measure Divergence (KL)**: For each image, compare its specific label distribution to the average distribution:

$$d_i = \mathrm{KL}(p(y|x_i)||\hat{p}(y))$$

$\rightarrow$ *We want the specific image to be distinct from the average soup.*

4. **Final Aggregation**: Average the divergences and exponentiate:

$$\mathrm{IS} = \exp\left(\frac{1}{N}\sum_{i=1}^{N} d_i\right)$$

Inception Score(IS)

Visualization

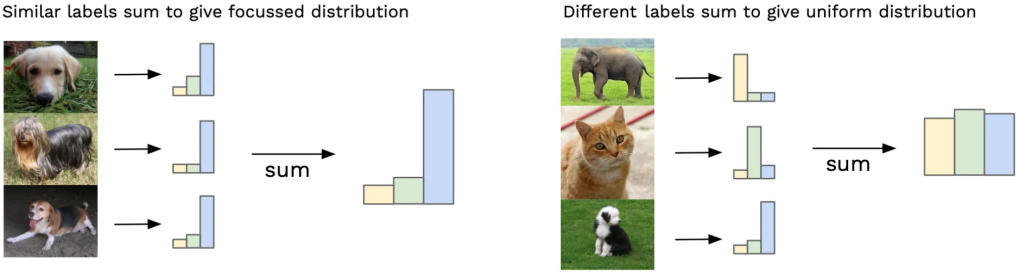


Figure: Changes in the output Probability distribution with change in the image generation. (source: medium)

What is CLIP?

Contrastive Language-Image Pre-training

The Big Idea:

- Joint vision-language model
- Learns to match images with text descriptions
- Trained on 400M image-text pairs
- Contrastive learning approach
- Shared embedding space for images and text

What is CLIP?

Contrastive Language-Image Pre-training

The Big Idea:

- Joint vision-language model
- Learns to match images with text descriptions
- Trained on 400M image-text pairs
- Contrastive learning approach
- Shared embedding space for images and text

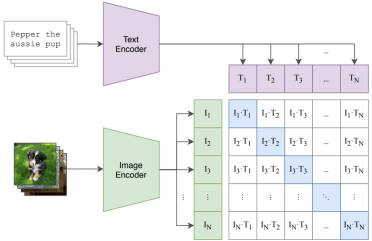


Figure: CLIP architecture(source : https://doi.org/10.48550/arXiv.2103.00020)

What is CLIP?

Contrastive Language-Image Pre-training

The Big Idea:

- Joint vision-language model
- Learns to match images with text descriptions
- Trained on 400M image-text pairs
- Contrastive learning approach
- Shared embedding space for images and text

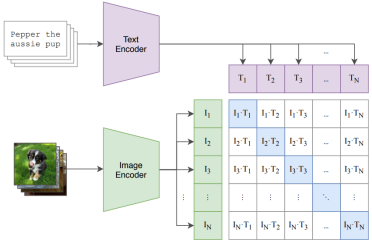


Figure: CLIP architecture(source : https://doi.org/10.48550/arXiv.2103.00020)

Training Objective

Maximize similarity between matching image-text pairs, minimize for non-matching pairs. By end of Training we get good Text and Vision encoders.

CLIP Score

Evaluating text-to-image generation

Motivation:

- Text-to-image models need caption alignment
- Traditional metrics (FID, IS) ignore text
- CLIP naturally measures image-text similarity

CLIP Score

Evaluating text-to-image generation

Motivation:

- Text-to-image models need caption alignment
- Traditional metrics (FID, IS) ignore text
- CLIP naturally measures image-text similarity

CLIP Score Definition:

$$\text{CLIPScore} = \max(100 \cdot \cos(E_I(x), E_T(c)), 0)$$

where:

- $E_I(x)$: CLIP image embedding
- $E_T(c)$: CLIP text embedding of caption c
- $\cos(\cdot)$: cosine similarity

CLIP Score

Evaluating text-to-image generation

Motivation:

- Text-to-image models need caption alignment
- Traditional metrics (FID, IS) ignore text
- CLIP naturally measures image-text similarity

CLIP Score Definition:

$$\text{CLIPScore} = \max(100 \cdot \cos(E_I(x), E_T(c)), 0)$$

where:

- $E_I(x)$: CLIP image embedding
- $E_T(c)$: CLIP text embedding of caption c
- $\cos(\cdot)$: cosine similarity

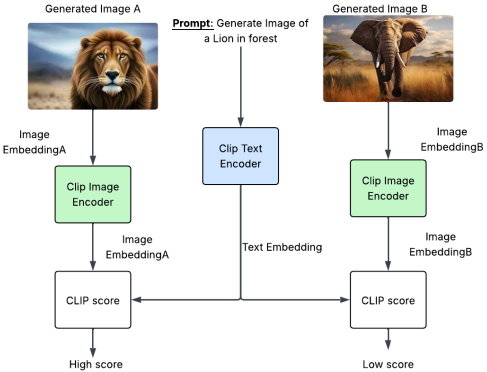


Figure: CLIP score pipeline

Comparing Evaluation Metrics

Choosing the right metric for your task

Metric	What it measures	Best for	Limitations
FID	Distribution similarity to real images	Overall quality & diversity of unconditional generation	Requires large sample size; sensitive to implementation
IS	Quality (sharpness) & diversity (variety)	Class-conditional generation	Ignores real data; biased to ImageNet
CLIP Score	Image-text alignment	Text-to-image generation	Doesn't measure realism alone; CLIP bias

Comparing Evaluation Metrics

Choosing the right metric for your task

Metric	What it measures	Best for	Limitations
FID	Distribution similarity to real images	Overall quality & diversity of unconditional generation	Requires large sample size; sensitive to implementation
IS	Quality (sharpness) & diversity (variety)	Class-conditional generation	Ignores real data; biased to ImageNet
CLIP Score	Image-text alignment	Text-to-image generation	Doesn't measure realism alone; CLIP bias

In Practice:

- Use **multiple metrics** together
- FID + IS for unconditional models
- FID + CLIP for text-conditional models
- Human evaluation still crucial

Comparing Evaluation Metrics

Choosing the right metric for your task

Metric	What it measures	Best for	Limitations
FID	Distribution similarity to real images	Overall quality & diversity of unconditional generation	Requires large sample size; sensitive to implementation
IS	Quality (sharpness) & diversity (variety)	Class-conditional generation	Ignores real data; biased to ImageNet
CLIP Score	Image-text alignment	Text-to-image generation	Doesn't measure realism alone; CLIP bias

In Practice:

- Use **multiple metrics** together
- FID + IS for unconditional models
- FID + CLIP for text-conditional models
- Human evaluation still crucial

Emerging Metrics:

- Precision & Recall (distribution coverage)
- Human Preference Scores
- Task-specific metrics
- Aesthetic predictors

The Diffusion phenomena

Introduction to Denoising Diffusion Probabilistic Models(DDPM)

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Physical Intuition:

- Consider a drop of ink falling into a glass of water.

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Physical Intuition:

- Consider a drop of ink falling into a glass of water.
- **Forward Process (Nature):** The ink naturally diffuses until it creates a uniform, chaotic distribution (Maximum Entropy / Pure Noise). This is easy and irreversible.

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Physical Intuition:

- Consider a drop of ink falling into a glass of water.
- **Forward Process (Nature):** The ink naturally diffuses until it creates a uniform, chaotic distribution (Maximum Entropy / Pure Noise). This is easy and irreversible.
- **Reverse Process (Generation):** "Rewinding time" to gather the ink back into a single drop.

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Core Insight:

- Destroying data structure is easy (add noise).

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Core Insight:

- Destroying data structure is easy (add noise).
- Creating data from scratch is hard.

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Core Insight:

- Destroying data structure is easy (add noise).
- Creating data from scratch is hard.
- **The Solution:** If we break the destruction into thousands of tiny steps, the reverse step becomes simple enough to learn.

Deep Unsupervised Learning via Nonequilibrium Thermodynamics

The "Origin Story" (Sohl-Dickstein et al., 2015)

The Core Insight:

- Destroying data structure is easy (add noise).
- Creating data from scratch is hard.
- **The Solution:** If we break the destruction into thousands of tiny steps, the reverse step becomes simple enough to learn.

Key Shift

One Hard Step (GAN)

vs

Many Easy Steps (Diffusion)

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.
- *End result:* Pure Gaussian noise (x_T).

2. The Reverse Process (p)

"The learned restoration."

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.
- *End result:* Pure Gaussian noise (x_T).

2. The Reverse Process (p)

"The learned restoration."

- We start with pure noise.

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.
- *End result:* Pure Gaussian noise (x_T).

2. The Reverse Process (p)

"The learned restoration."

- We start with pure noise.
- The Neural Network tries to "denoise" it one step at a time.

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.
- *End result:* Pure Gaussian noise (x_T).

2. The Reverse Process (p)

"The learned restoration."

- We start with pure noise.
- The Neural Network tries to "denoise" it one step at a time.
- **Key:** This is the **trainable** part.

Intuition: The Forward and Reverse Processes

The process involved in Diffusion models consists of two approaches:

1. The Forward Process (q)

"The systematic destruction of data."

- We slowly add Gaussian noise to an image.
- **Key:** This is a **fixed** process. There is no learning here. We just follow a schedule.
- *End result:* Pure Gaussian noise (x_T).

2. The Reverse Process (p)

"The learned restoration."

- We start with pure noise.
- The Neural Network tries to "denoise" it one step at a time.
- **Key:** This is the **trainable** part.

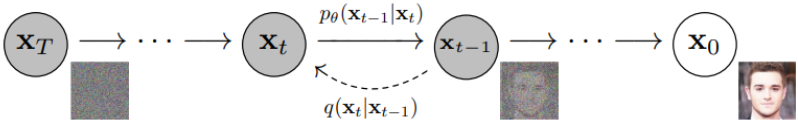


Figure: Forward and Reverse proccesses illustration. (source : DDPM paper)

The Forward Diffusion Process

Gradually adding noise to images

The Big Picture:

- Start with a real image x_0
- Gradually add noise over T steps
- End with pure random noise x_T
- This is a **fixed process** (not learned!)

Key Property: Markov Chain

- Each step depends *only* on the previous step
- We don't need to remember the entire history

The Forward Diffusion

Step 1: The Joint Probability (Markov Chain)

The probability of the entire sequence:

$$q(x_{1:T}|x_0) = q(x_1|x_0) \cdot q(x_2|x_1) \cdot q(x_3|x_2) \cdots q(x_T|x_{T-1}) = \prod_{t=1}^{T} q(x_t|x_{t-1})$$

Each term depends only on the previous step (Markov property)

The Forward Diffusion Process

How do we add noise at each step?

Step 2: The Noise Addition Formula

At each time step t, we add Gaussian (normal) noise:

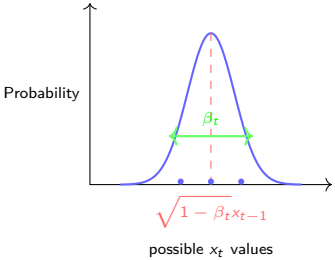
$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t \mathbf{I})$$

Let's break this down:

What does this mean?

x_t is drawn from a Gaussian with:

- **Mean:** $\sqrt{1-\beta_t}\, x_{t-1}$
  - $\rightarrow$ Previous image, slightly scaled down
- **Variance:** $\beta_t \mathbf{I}$
  - $\rightarrow$ Amount of noise added



Reparameterization Trick: Motivation

Can we skip to any timestep directly?

The Problem:

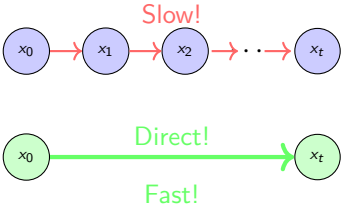
- To get x_{500}, we need to compute:

$$x_0 \to x_1 \to x_2 \to \cdots$$
$$\to x_{499} \to x_{500}$$

- That's 500 sequential operations!
- Very slow for training

What we want:

- Jump directly: $x_0 \to x_t$ in one step
- For *any* timestep t



Key Insight

Since we're adding Gaussian noise at each step, and **sums of Gaussians are Gaussian**, we can derive a closed-form formula to jump directly from x_0 to x_t!

Reparameterization Trick: Derivation

Step-by-step mathematical derivation

Let's define some notation first:

Notation

- $\alpha_t = 1 - \beta_t$ (complement of noise)
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdot \alpha_3 \cdots \alpha_t$ (cumulative product)

Reparameterization Trick: Derivation

Step-by-step mathematical derivation

Let's define some notation first:

Notation

- $\alpha_t = 1 - \beta_t$ (complement of noise)
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdot \alpha_3 \cdots \alpha_t$ (cumulative product)

Step 1: Recall the one-step formula with Gaussian reparameterization:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1 - \alpha_t}\, \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$$

Reparameterization Trick: Derivation

Step-by-step mathematical derivation

Let's define some notation first:

Notation

- $\alpha_t = 1 - \beta_t$ (complement of noise)
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdot \alpha_3 \cdots \alpha_t$ (cumulative product)

Step 1: Recall the one-step formula with Gaussian reparameterization:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1 - \alpha_t}\, \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$$

Step 2: Let's expand x_t by substituting x_{t-1}:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1 - \alpha_t}\, \epsilon_{t-1}$$

Reparameterization Trick: Derivation

Step-by-step mathematical derivation

Let's define some notation first:

Notation

- $\alpha_t = 1 - \beta_t$ (complement of noise)
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdot \alpha_3 \cdots \alpha_t$ (cumulative product)

Step 1: Recall the one-step formula with Gaussian reparameterization:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$$

Step 2: Let's expand x_t by substituting x_{t-1}:

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \\ &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-2}\right) + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \end{aligned}$$

Reparameterization Trick: Derivation

Step-by-step mathematical derivation

Let's define some notation first:

Notation

- $\alpha_t = 1 - \beta_t$ (complement of noise)
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdot \alpha_3 \cdots \alpha_t$ (cumulative product)

Step 1: Recall the one-step formula with Gaussian reparameterization:

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}, \quad \epsilon_{t-1} \sim \mathcal{N}(0, \mathbf{I})$$

Step 2: Let's expand x_t by substituting x_{t-1}:

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \\ &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-2}\right) + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \\ &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \end{aligned}$$

Reparameterization Trick: Derivation (cont.)

Combining the noise terms

Step 3: From the previous slide, we had:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1 - \alpha_t}\, \epsilon_{t-1}$$

Reparameterization Trick: Derivation (cont.)

Combining the noise terms

Step 3: From the previous slide, we had:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t (1 - \alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1 - \alpha_t}\, \epsilon_{t-1}$$

Key fact: Two independent Gaussians can be combined:

$$\sqrt{a}\, \epsilon_1 + \sqrt{b}\, \epsilon_2 = \sqrt{a + b}\, \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Reparameterization Trick: Derivation (cont.)

Combining the noise terms

Step 3: From the previous slide, we had:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}$$

Key fact: Two independent Gaussians can be combined:

$$\sqrt{a}\, \epsilon_1 + \sqrt{b}\, \epsilon_2 = \sqrt{a+b}\, \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

So the red terms combine:

$$\begin{aligned}
&\sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \\
&= \sqrt{\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t)}\, \epsilon \\
&= \sqrt{\alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t}\, \epsilon \\
&= \sqrt{1 - \alpha_t\alpha_{t-1}}\, \epsilon
\end{aligned}$$

Reparameterization Trick: Derivation (cont.)

Combining the noise terms

Step 3: From the previous slide, we had:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1}$$

Key fact: Two independent Gaussians can be combined:

$$\sqrt{a}\, \epsilon_1 + \sqrt{b}\, \epsilon_2 = \sqrt{a+b}\, \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

So the red terms combine:

$$\begin{aligned}
&\sqrt{\alpha_t(1-\alpha_{t-1})}\, \epsilon_{t-2} + \sqrt{1-\alpha_t}\, \epsilon_{t-1} \\
&= \sqrt{\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t)}\, \epsilon \\
&= \sqrt{\alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t}\, \epsilon \\
&= \sqrt{1 - \alpha_t\alpha_{t-1}}\, \epsilon
\end{aligned}$$

Therefore:

$$x_t = \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\, \epsilon$$

Reparameterization Trick: Final Formula

The closed-form solution

Step 4: Continue this process all the way back to x_0:

$$
\begin{aligned}
x_t &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}}\, \epsilon \\
&= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}}\, x_{t-3} + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2}}\, \epsilon \\
&\vdots \\
&= \sqrt{\alpha_t \alpha_{t-1} \cdots \alpha_1}\, x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \cdots \alpha_1}\, \epsilon
\end{aligned}
$$

Reparameterization Trick: Final Formula

The closed-form solution

Step 4: Continue this process all the way back to x_0:

$$\begin{aligned} x_t &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}}\, \epsilon \\ &= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}}\, x_{t-3} + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2}}\, \epsilon \\ &\vdots \\ &= \sqrt{\alpha_t \alpha_{t-1} \cdots \alpha_1}\, x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \cdots \alpha_1}\, \epsilon \end{aligned}$$

Using our notation: $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \cdot \alpha_2 \cdots \alpha_t$

The Reparameterization Formula

$$\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})}$$

Or equivalently: $q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t)\mathbf{I})$

Reparameterization Trick: Advantages

The Reparameterization Formula

$$\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})}$$

Or equivalently: $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1 - \bar{\alpha}_t)\mathbf{I})$

Why This Is Powerful

- Jump directly from x_0 to x_t in **one step**!
- Just sample $\epsilon \sim \mathcal{N}(0, \mathbf{I})$ and compute x_t
- Essential for efficient training

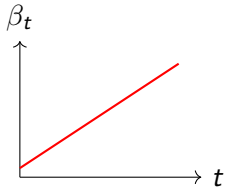
Noise Schedulers

Controlling the rate of diffusion

The behavior of the diffusion depends on the **Variance Schedule** β_t.

Constraints:

- $0 < \beta_t < 1$ for all t.
- Typically $\beta_1 < \beta_2 < \cdots < \beta_T$ (noise increases over time).



Linear Schedule Example

Standard Linear Schedule:

- $T = 1000$ steps.
- Linear increase from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$.
- $\beta_1 = 0.0001 \rightarrow \beta_{1000} = 0.02$.

Impact on Signal

As β_t increases, $\bar{\alpha}_t \rightarrow 0$. By step T, $x_T \approx \mathcal{N}(0, \mathbf{I})$ (Pure Noise).

The Reverse Denoising Process

The Goal: Learning to "Undo" the Noise

The Problem:

- We have pure noise $x_T \sim \mathcal{N}(0, \mathbf{I})$.
- We want to generate a clean image x_0.
- The forward process $q(x_t|x_{t-1})$ was a Gaussian.

The Solution:

- If the forward steps are small enough, the reverse step is *also* Gaussian.
- We define a parameterized model (Neural Network) to approximate this reverse transition:

The Reverse Denoising Process

The Goal: Learning to "Undo" the Noise

The Model Definition

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(t))$$

- **Input:** Noisy image x_t and timestep t.
- **Output:** The mean μ_θ of the slightly less noisy image x_{t-1}.
- **Note:** We usually keep the variance Σ_θ fixed to a constant (like β_t) for simplicity.

Deriving the Ground Truth (Step 1)

The Intractable vs. The Tractable

To train our model p_θ, we need a "Ground Truth" target to compare it against.

The Challenge: Calculating the true reverse step $q(x_{t-1}|x_t)$ directly from noise is impossible (it would require knowing the distribution of all possible images).

Deriving the Ground Truth (Step 1)

The Intractable vs. The Tractable

To train our model p_θ, we need a "Ground Truth" target to compare it against.

The Challenge: Calculating the true reverse step $q(x_{t-1}|x_t)$ directly from noise is impossible (it would require knowing the distribution of all possible images).

The Workaround (Conditioning on x_0): However, if we **know the original image** x_0 (which we do during training), the reverse step becomes tractable!
We define the posterior:

$$q(x_{t-1}|x_t, x_0)$$

This distribution tells us: "Given where we started (x_0) and where we are (x_t), where must we have been at the previous step (x_{t-1})?".

Deriving the Ground Truth (Step 2)

Applying Bayes' Rule

We calculate $q(x_{t-1}|x_t, x_0)$ using Bayes' Rule:

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0) \cdot q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

Since the forward process is Markovian (x_t depends only on x_{t-1}), we can simplify $q(x_t|x_{t-1}, x_0)$ to just $q(x_t|x_{t-1})$.

Substituting the Gaussian definitions we derived in the Forward Process:

$$= \frac{\mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t \mathbf{I}) \cdot \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})\mathbf{I})}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})}$$

Result: The product of Gaussians is a Gaussian.

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

Deriving the Ground Truth (Step 3)

The Target Mean

After simplifying the algebra, the mean of this ground truth distribution $\tilde{\mu}_t$ is:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

Why is this equation important?

- It tells us exactly what the mean of the image *should* be at the previous step.
- Notice that x_t is available to the network.
- The only unknown part is ϵ (the noise added to get to x_t).

Deriving the Ground Truth (Step 3)

The Target Mean

After simplifying the algebra, the mean of this ground truth distribution $\tilde{\mu}_t$ is:

$$\tilde{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

Why is this equation important?

- It tells us exactly what the mean of the image *should* be at the previous step.
- Notice that x_t is available to the network.
- The only unknown part is ϵ (the noise added to get to x_t).

Key Insight

To predict the previous image mean $\tilde{\mu}_t$, the neural network only needs to predict the noise ϵ.

Simplified Loss Function

From Distributions to Mean Squared Error

1. The Objective: We want our model distribution p_θ to be close to the ground truth posterior q. We measure this with KL Divergence:

$$L = D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$$

Simplified Loss Function

From Distributions to Mean Squared Error

1. The Objective: We want our model distribution p_θ to be close to the ground truth posterior q. We measure this with KL Divergence:

$$L = D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$$

2. Simplification: Since both distributions are Gaussian with fixed variance, minimizing KL Divergence is mathematically equivalent to minimizing the distance between their means:

$$L \approx ||\tilde{\mu}_q(t) - \mu_\theta(x_t, t)||^2$$

Simplified Loss Function

From Distributions to Mean Squared Error

1. The Objective: We want our model distribution p_θ to be close to the ground truth posterior q. We measure this with KL Divergence:

$$L = D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t))$$

2. Simplification: Since both distributions are Gaussian with fixed variance, minimizing KL Divergence is mathematically equivalent to minimizing the distance between their means:

$$L \approx ||\tilde{\mu}_q(t) - \mu_\theta(x_t, t)||^2$$

3. The Reparameterization: Instead of the network outputting μ_θ, we train it to output the noise $\epsilon_\theta(x_t, t)$. Substituting the formula for $\tilde{\mu}$ from the previous slide, the messy constants cancel out, leaving us with a simple objective.

The Final Training Objective

Simple, Elegant, Effective

The complex derivation boils down to this simple loss function used in implementation:

$$\mathcal{L}_{\text{simple}} = ||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon}_{\text{Noisy Input } x_t}, t)||^2$$

Interpretation:

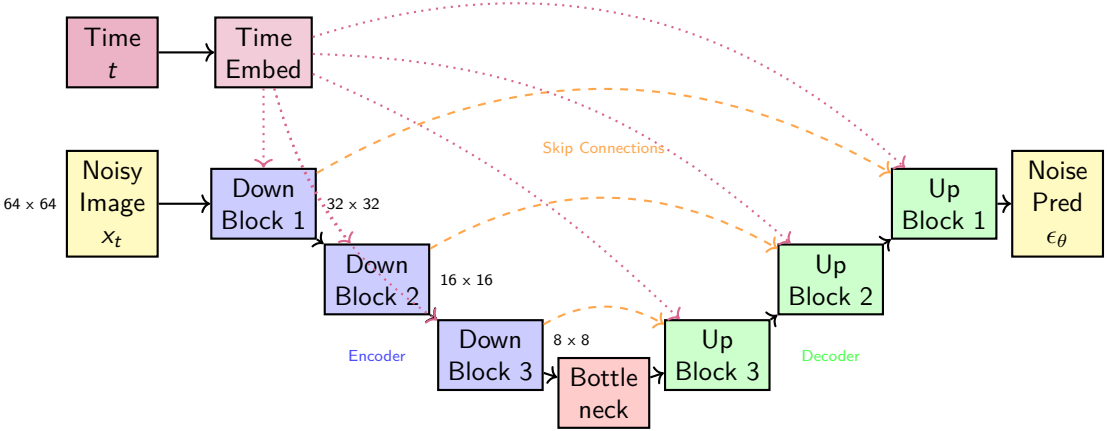
1. Take a clean image x_0.
2. Sample random noise ϵ.
3. Add the noise to create x_t (using the Forward Process).
4. Ask the Neural Network (U-Net): **"What noise was added to this image?"**
5. Calculate MSE between the *actual noise* (ϵ) and *predicted noise* (ϵ_θ).

We are literally teaching the model to separate signal from noise.

DDPM Model Architecture Overview

U-Net with time conditioning

The Model: A modified U-Net architecture



DDPM Model Architecture Overview

U-Net with time Conditioning

Key Components

- **Encoder**: Downsamples image, extracts features
- **Decoder**: Upsamples features, reconstructs spatial dimensions
- **Skip Connections**: Preserve fine details from encoder
- **Time Embedding**: Injects timestep information at every level

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!
- Need a representation that captures this similarity

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!
- Need a representation that captures this similarity

The Solution:

- Convert timestep to a high-dimensional vector

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!
- Need a representation that captures this similarity

The Solution:

- Convert timestep to a high-dimensional vector
- Use sinusoidal embeddings (like in Transformers)

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!
- Need a representation that captures this similarity

The Solution:

- Convert timestep to a high-dimensional vector
- Use sinusoidal embeddings (like in Transformers)
- Nearby timesteps get similar embeddings

Time Embeddings: The Challenge

How do we tell the network what time it is?

The Problem:

- Timestep is just a number: $t \in \{1, 2, \ldots, T\}$
- If we use it directly: $t = 500$ vs $t = 501$ look too different
- But these timesteps should be *similar*!
- Need a representation that captures this similarity

The Solution:

- Convert timestep to a high-dimensional vector
- Use sinusoidal embeddings (like in Transformers)
- Nearby timesteps get similar embeddings

Key Idea

Similar timesteps should have similar representations. Embeddings create a smooth, continuous space where the network can learn temporal patterns.

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Properties:

- Different dimensions use different frequencies

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Properties:

- Different dimensions use different frequencies
- Low dimensions: fast oscillations (capture fine differences)

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Properties:

- Different dimensions use different frequencies
- Low dimensions: fast oscillations (capture fine differences)
- High dimensions: slow oscillations (capture coarse patterns)

Sinusoidal Time Embeddings

The mathematical formulation

Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Properties:

- Different dimensions use different frequencies
- Low dimensions: fast oscillations (capture fine differences)
- High dimensions: slow oscillations (capture coarse patterns)
- Bounded values: $[-1, 1]$

Sinusoidal Time Embeddings

The mathematical formulation

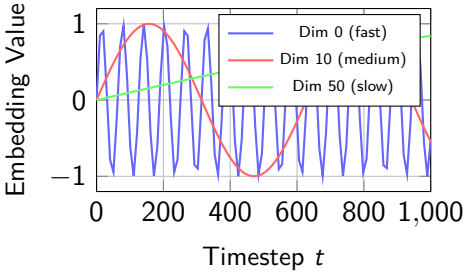
Inspired by Transformer positional encodings

For a timestep t and embedding dimension d, we compute:

$$\text{emb}(t)_i = \begin{cases} \sin\left(\frac{t}{10000^{2i/d}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{t}{10000^{(2i-1)/d}}\right) & \text{if } i \text{ is odd} \end{cases}$$

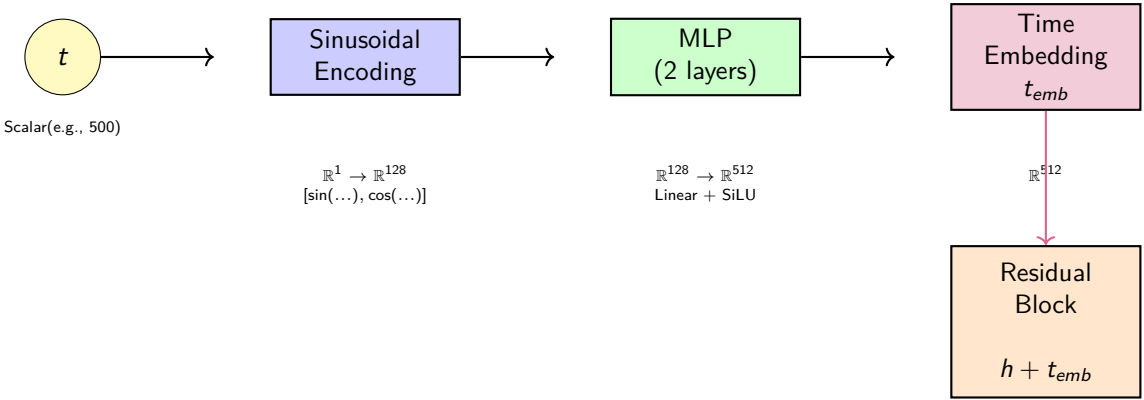
Properties:

- Different dimensions use different frequencies
- Low dimensions: fast oscillations (capture fine differences)
- High dimensions: slow oscillations (capture coarse patterns)
- Bounded values: $[-1, 1]$
- Deterministic (no learning needed)



Time Embedding Pipeline

From timestep to network conditioning



Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task
- Two linear layers with SiLU activation

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task
- Two linear layers with SiLU activation

How It's Used:

- t_{emb} broadcast to spatial dimensions

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task
- Two linear layers with SiLU activation

How It's Used:

- t_{emb} broadcast to spatial dimensions
- Added to feature maps in each block

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task
- Two linear layers with SiLU activation

How It's Used:

- t_{emb} broadcast to spatial dimensions
- Added to feature maps in each block
- Modulates the network's behavior

Time Embedding Pipeline

From timestep to network conditioning

Why MLP After Sinusoidal?

- Sinusoidal gives fixed encoding
- MLP learns task-specific representation
- Adapts embedding to denoising task
- Two linear layers with SiLU activation

How It's Used:

- t_{emb} broadcast to spatial dimensions
- Added to feature maps in each block
- Modulates the network's behavior
- Same embedding used at all spatial locations

Model Architecture Comparison

DDPM choices vs alternatives

Component	DDPM Choice	Why?
Base Architecture	U-Net	Excellent for image-to-image tasks; preserves spatial info
Time Conditioning	Sinusoidal + MLP added to features	Smooth temporal representation; used in Transformers
Normalization	Group Normalization	Works better than BatchNorm for small batch sizes
Activation	SiLU (Swish)	Smoother gradients than ReLU
Skip Connections	Concatenation	Preserves fine details from encoder
Attention	Self-attention at 16 x 16 resolution	Captures long-range dependencies
Prediction Target	Noise ϵ	More stable than predicting x_0

Algorithm 1: Training

1. **repeat**
2. Sample clean image $x_0 \sim q(x_{data})$
3. Sample timestep $t \sim \text{Uniform}(\{1, \ldots, T\})$
4. Sample noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
5. Take gradient descent step on:

$$\nabla_\theta ||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon}_{\text{Noisy Input } x_t}, t)||^2$$

6. **until** converged

Implementation - Training Algorithm

Key Explanations:

- **Random Timestep t:** We don't generate a full chain during training. We pick *one* random moment in the diffusion process and ask the model to fix it. This makes training efficient.

Implementation - Training Algorithm

Key Explanations:

- **Random Timestep t:** We don't generate a full chain during training. We pick *one* random moment in the diffusion process and ask the model to fix it. This makes training efficient.
- **Input to ϵ_θ:** The network receives the noisy image (x_t) and the time embedding (t).

Implementation - Training Algorithm

Key Explanations:

- **Random Timestep t:** We don't generate a full chain during training. We pick *one* random moment in the diffusion process and ask the model to fix it. This makes training efficient.
- **Input to ϵ_θ:** The network receives the noisy image (x_t) and the time embedding (t).
- **Objective:** Minimize the distance between the *actual* noise added (ϵ) and the *predicted* noise (ϵ_θ).

Algorithm 2: Sampling (The Reverse Process)

1. Sample $x_T \sim \mathcal{N}(0, \mathbf{I})$ (Start with pure noise)
2. **for** $t = T, \ldots, 1$ **do**
3. Sample $z \sim \mathcal{N}(0, \mathbf{I})$ if $t > 1$, else $z = 0$
4. Compute mean $\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$
5. Update image: $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z$
6. **end for**
7. **return** x_0

Implementation - Inference Algorithm

Key Explanations:

- **Iterative Refinement:** We cannot jump from $T \to 0$. We must peel off the noise layer by layer.

Implementation - Inference Algorithm

Key Explanations:

- **Iterative Refinement:** We cannot jump from $T \to 0$. We must peel off the noise layer by layer.
- **The Mean Calculation:** This formula uses the predicted noise ϵ_θ to estimate the center of the previous distribution.

Implementation - Inference Algorithm

Key Explanations:

- **Iterative Refinement:** We cannot jump from $T \to 0$. We must peel off the noise layer by layer.
- **The Mean Calculation:** This formula uses the predicted noise ϵ_θ to estimate the center of the previous distribution.
- **The Noise Term (z):** We add a small amount of random noise back in at every step (except the last). This injects stochasticity (Langevin dynamics), helping the model generate diverse samples rather than collapsing to a single average image.

Accelerating the Generations

Introduction to DDIM

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.
- **Strict Dependency:** To calculate step 999, we *must* have the result of step 1000. We cannot parallelize this.

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.
- **Strict Dependency:** To calculate step 999, we *must* have the result of step 1000. We cannot parallelize this.

The Cost Calculation:

- A typical U-Net is large (millions of parameters).

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.
- **Strict Dependency:** To calculate step 999, we *must* have the result of step 1000. We cannot parallelize this.

The Cost Calculation:

- A typical U-Net is large (millions of parameters).
- DDPM requires $T = 1000$ steps.

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.
- **Strict Dependency:** To calculate step 999, we *must* have the result of step 1000. We cannot parallelize this.

The Cost Calculation:

- A typical U-Net is large (millions of parameters).
- DDPM requires $T = 1000$ steps.
- **Result:** Generating **one image** requires running the heavy neural network **1000 times**.

The Computational Bottleneck of DDPM

Why is it so slow?

The Markov Chain Limit:

- Recall the Reverse Process: We move from $T \to T-1 \to T-2 \cdots \to 0$.
- **Strict Dependency:** To calculate step 999, we *must* have the result of step 1000. We cannot parallelize this.

The Cost Calculation:

- A typical U-Net is large (millions of parameters).
- DDPM requires $T = 1000$ steps.
- **Result:** Generating **one image** requires running the heavy neural network **1000 times**.

The Problem

High Quality, but Low Speed.
(Minutes per image vs. Milliseconds)

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

- **DDPM (Stochastic):** x_{t-1} is drawn from a random distribution around x_t.

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

- **DDPM (Stochastic):** x_{t-1} is drawn from a random distribution around x_t.
- **DDIM (Deterministic):** We can redefine the noise schedule such that the mapping from noise to image is **deterministic** (not random), yet still arrives at the same training objective.

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

- **DDPM (Stochastic):** x_{t-1} is drawn from a random distribution around x_t.
- **DDIM (Deterministic):** We can redefine the noise schedule such that the mapping from noise to image is **deterministic** (not random), yet still arrives at the same training objective.

Why does this help?

- Because the path is deterministic, the "trajectory" from Noise $\rightarrow$ Image is smooth and predictable.

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

- **DDPM (Stochastic):** x_{t-1} is drawn from a random distribution around x_t.
- **DDIM (Deterministic):** We can redefine the noise schedule such that the mapping from noise to image is **deterministic** (not random), yet still arrives at the same training objective.

Why does this help?

- Because the path is deterministic, the "trajectory" from Noise $\rightarrow$ Image is smooth and predictable.
- We don't need to take baby steps. We can take **huge strides**.

DDIM: Denoising Diffusion Implicit Models

The "Zero-Shot" Speedup

The Core Insight: Researchers (Song et al.) asked: *"Does the forward process HAVE to be random?"*

- **DDPM (Stochastic):** x_{t-1} is drawn from a random distribution around x_t.
- **DDIM (Deterministic):** We can redefine the noise schedule such that the mapping from noise to image is **deterministic** (not random), yet still arrives at the same training objective.

Why does this help?

- Because the path is deterministic, the "trajectory" from Noise $\rightarrow$ Image is smooth and predictable.
- We don't need to take baby steps. We can take **huge strides**.

Key Benefit: We can use the **exact same pre-trained DDPM model**. No retraining required!

DDPM vs. DDIM Inference

Skipping Steps

DDPM Sampling:

- Must visit every integer: $1000, 999, 998 \ldots 0$.

DDPM vs. DDIM Inference

Skipping Steps

DDPM Sampling:

- Must visit every integer: $1000, 999, 998 \ldots 0$.
- Total: 1000 steps.

DDPM vs. DDIM Inference

Skipping Steps

DDPM Sampling:

- Must visit every integer: $1000, 999, 998 \ldots 0$.
- Total: 1000 steps.
- Noise is injected at every step ($z \sim \mathcal{N}$).

DDIM Sampling:

- Can select a subset of steps: $\tau = [1000, 950, 900 \ldots 0]$.

DDPM vs. DDIM Inference

Skipping Steps

DDPM Sampling:

- Must visit every integer: $1000, 999, 998 \ldots 0$.
- Total: 1000 steps.
- Noise is injected at every step ($z \sim \mathcal{N}$).

DDIM Sampling:

- Can select a subset of steps: $\tau = [1000, 950, 900 \ldots 0]$.
- Total: 20 steps.
- Deterministic calculation (Equation is an ODE discretization).

DDPM vs. DDIM Inference

Skipping Steps

DDPM Sampling:

- Must visit every integer: $1000, 999, 998 \ldots 0$.
- Total: 1000 steps.
- Noise is injected at every step ($z \sim \mathcal{N}$).

DDIM Sampling:

- Can select a subset of steps: $\tau = [1000, 950, 900 \ldots 0]$.
- Total: 20 steps.
- Deterministic calculation (Equation is an ODE discretization).

Method	Steps needed for Good Quality	Time (approx)
DDPM	1000	20 sec
DDIM	**20 - 50**	**1 sec**

DDIM allows us to trade a tiny bit of quality for a massive gain in speed.

Skipping the Steps

- **Problem:** DDPM requires many steps (1000+) for generation

Skipping the Steps

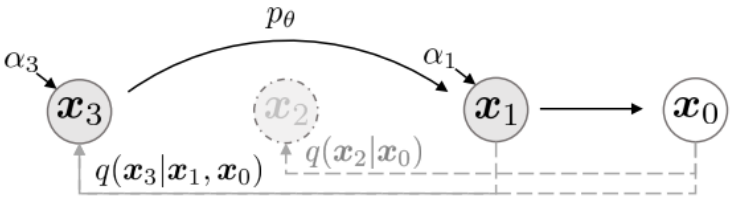
- **Problem:** DDPM requires many steps (1000+) for generation
- Each step: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$

Skipping the Steps

- **Problem:** DDPM requires many steps (1000+) for generation
- Each step: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$
- **Key insight:** Do we need to visit *every* timestep?

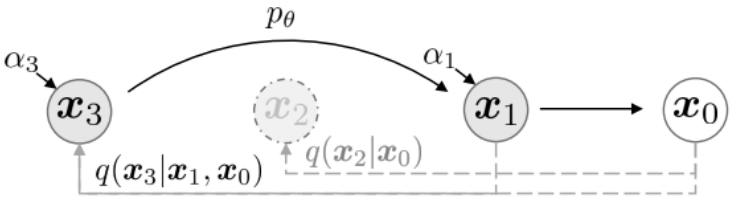
Skipping the Steps

- **Problem:** DDPM requires many steps (1000+) for generation
- Each step: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$
- **Key insight:** Do we need to visit *every* timestep?
- DDIM says: No! We can skip steps and still get good results



Skipping the Steps

- **Problem:** DDPM requires many steps (1000+) for generation
- Each step: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$
- **Key insight:** Do we need to visit *every* timestep?
- DDIM says: No! We can skip steps and still get good results



Speed-up

Generate in 10-50 steps instead of 1000 steps -> **20-100 x faster!**

DDIM Update Rule

Instead of going step-by-step, DDIM jumps directly:

DDIM Sampling Formula

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

DDIM Update Rule

Instead of going step-by-step, DDIM jumps directly:

DDIM Sampling Formula

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

Breaking it down:

1. Predict the clean image x_0 from current noisy x_t

DDIM Update Rule

Instead of going step-by-step, DDIM jumps directly:

DDIM Sampling Formula

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

Breaking it down:

1. Predict the clean image x_0 from current noisy x_t
2. Use x_0 prediction to jump to any previous timestep $t-1$

DDIM Update Rule

Instead of going step-by-step, DDIM jumps directly:

DDIM Sampling Formula

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

Breaking it down:

1. Predict the clean image x_0 from current noisy x_t
2. Use x_0 prediction to jump to any previous timestep $t-1$
3. No randomness -> deterministic path

DDIM Update Rule

Instead of going step-by-step, DDIM jumps directly:

DDIM Sampling Formula

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)}_{\text{direction pointing to } x_t}$$

Breaking it down:

1. Predict the clean image x_0 from current noisy x_t
2. Use x_0 prediction to jump to any previous timestep $t-1$
3. No randomness -> deterministic path

Timestep subsequence: Choose $\tau_1, \tau_2, ..., \tau_S$ where $S \ll T$

- Example: $[1, 10, 20, ..., 1000]$ instead of $[1, 2, 3, ..., 1000]$

Understanding the Formula Intuitively

DDIM Formula (Simplified View)

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

Understanding the Formula Intuitively

DDIM Formula (Simplified View)

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

Think of it as a weighted combination:

- $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}$ is our predicted clean image

Understanding the Formula Intuitively

DDIM Formula (Simplified View)

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

Think of it as a weighted combination:

- $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}$ is our predicted clean image
- $\epsilon_\theta(x_t, t)$ is the predicted noise

Understanding the Formula Intuitively

DDIM Formula (Simplified View)

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

Think of it as a weighted combination:

- $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}$ is our predicted clean image
- $\epsilon_\theta(x_t, t)$ is the predicted noise
- We mix clean image and noise according to the noise schedule at $t - 1$

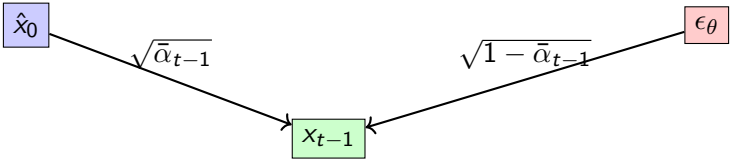
Understanding the Formula Intuitively

DDIM Formula (Simplified View)

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \cdot \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \cdot \epsilon_\theta(x_t, t)$$

Think of it as a weighted combination:

- $\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t,t)}{\sqrt{\bar{\alpha}_t}}$ is our predicted clean image
- $\epsilon_\theta(x_t, t)$ is the predicted noise
- We mix clean image and noise according to the noise schedule at $t - 1$



Semantics in Noise (DDIM Inversion)

Big Question: Can we go backwards? From image -> noise?

Semantics in Noise (DDIM Inversion)

Big Question: Can we go backwards? From image -> noise?

Why would we want this?

- **Image editing:** Start with real image, convert to noise, edit, denoise

Semantics in Noise (DDIM Inversion)

Big Question: Can we go backwards? From image -> noise?

Why would we want this?

- **Image editing:** Start with real image, convert to noise, edit, denoise
- **Understanding:** What does the model "see" in the noise?

Semantics in Noise (DDIM Inversion)

Big Question: Can we go backwards? From image -> noise?

Why would we want this?

- **Image editing:** Start with real image, convert to noise, edit, denoise
- **Understanding:** What does the model "see" in the noise?
- **Interpolation:** Blend images in latent space

DDIM Inversion

Since DDIM is deterministic, we can reverse the process exactly!

$$\text{Real Image } x_0 \xrightarrow{\text{DDIM Inversion}} \text{Noise } x_T \xrightarrow{\text{DDIM Sampling}} \text{Reconstructed } x_0$$

Semantics in Noise (DDIM Inversion)

Big Question: Can we go backwards? From image -> noise?

Why would we want this?

- **Image editing:** Start with real image, convert to noise, edit, denoise
- **Understanding:** What does the model "see" in the noise?
- **Interpolation:** Blend images in latent space

DDIM Inversion

Since DDIM is deterministic, we can reverse the process exactly!

$$\text{Real Image } x_0 \xrightarrow{\text{DDIM Inversion}} \text{Noise } x_T \xrightarrow{\text{DDIM Sampling}} \text{Reconstructed } x_0$$

Key property: The noise x_T contains semantic information about x_0

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

Process:

1. Start with real image x_0

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

Process:

1. Start with real image x_0
2. Predict what noise would have created it: $\epsilon_\theta(x_0, 0)$

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

Process:

1. Start with real image x_0
2. Predict what noise would have created it: $\epsilon_\theta(x_0, 0)$
3. Use this to compute $x_1, x_2, ..., x_T$

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

Process:

1. Start with real image x_0
2. Predict what noise would have created it: $\epsilon_\theta(x_0, 0)$
3. Use this to compute $x_1, x_2, ..., x_T$
4. End with noise x_T that encodes the image

DDIM Inversion: The Math

Forward (Inversion): Go from x_{t-1} to x_t

DDIM Inversion Formula

$$x_t = \sqrt{\bar{\alpha}_t} \underbrace{\left(\frac{x_{t-1} - \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_{t-1}, t-1)}{\sqrt{\bar{\alpha}_{t-1}}} \right)}_{\text{predicted } x_0} + \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_{t-1}, t-1)$$

Process:

1. Start with real image x_0
2. Predict what noise would have created it: $\epsilon_\theta(x_0, 0)$
3. Use this to compute $x_1, x_2, ..., x_T$
4. End with noise x_T that encodes the image

Important

This works because DDIM is deterministic. DDPM cannot do this due to randomness!

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Example: Image Editing

1. Invert photo of a cat: $x_0^{\text{cat}} \to x_T^{\text{cat}}$

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Example: Image Editing

1. Invert photo of a cat: $x_0^{\text{cat}} \to x_T^{\text{cat}}$
2. Change text prompt to "dog"

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Example: Image Editing

1. Invert photo of a cat: $x_0^{\text{cat}} \to x_T^{\text{cat}}$
2. Change text prompt to "dog"
3. Denoise x_T^{cat} with "dog" prompt

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Example: Image Editing

1. Invert photo of a cat: $x_0^{\text{cat}} \to x_T^{\text{cat}}$
2. Change text prompt to "dog"
3. Denoise x_T^{cat} with "dog" prompt
4. Result: dog in same pose/composition as original cat!

Why "Semantics in Noise"?

The inverted noise x_T is not random, it's **semantically meaningful**!

Regular Random Noise:

- No structure
- Any image possible
- Start fresh

DDIM Inverted Noise:

- Encodes original image
- Specific trajectory
- Enables editing

Example: Image Editing

1. Invert photo of a cat: $x_0^{\text{cat}} \to x_T^{\text{cat}}$
2. Change text prompt to "dog"
3. Denoise x_T^{cat} with "dog" prompt
4. Result: dog in same pose/composition as original cat!

The noise preserves: composition, structure, layout

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure
- Same training: uses DDPM models

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure
- Same training: uses DDPM models

Limitations

- Less diversity: no randomness

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure
- Same training: uses DDPM models

Limitations

- Less diversity: no randomness
- Inversion not perfect: small errors accumulate

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure
- Same training: uses DDPM models

Limitations

- Less diversity: no randomness
- Inversion not perfect: small errors accumulate
- Quality vs speed tradeoff

DDIM: Pros and Cons

Advantages

- Fast: 10-50 steps
- Deterministic: same result every time
- Invertible: can go image -> noise
- Enables editing: preserve structure
- Same training: uses DDPM models

Limitations

- Less diversity: no randomness
- Inversion not perfect: small errors accumulate
- Quality vs speed tradeoff

When to use DDIM?

- Need fast generation
- Image editing tasks
- Reproducible results
- Interpolation between images

Summary: DDIM Key Takeaways

1. **Speed-up:** Skip timesteps -> 20-100 x faster than DDPM
2. **Deterministic:** Remove randomness $\rightarrow$ same input = same output
3. **Inversion:** Go backwards image -> noise deterministically
4. **Semantic noise:** Inverted noise encodes image structure
5. **Applications:** Image editing, interpolation, fast sampling

The Big Idea

DDIM discovers that diffusion models don't need randomness.
The deterministic trajectory enables **both speed and control**.

Paper: Denoising Diffusion Implicit Models (Song et al., 2020)

Guidance and Controlled generation

Conditioning with Text and Image

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

- Generate only dogs

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

- Generate only dogs
- Generate cats with specific features

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

- Generate only dogs
- Generate cats with specific features
- Generate images matching text: "a dog wearing sunglasses"

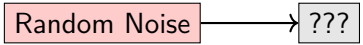
Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

- Generate only dogs
- Generate cats with specific features
- Generate images matching text: "a dog wearing sunglasses"



No control!

Conditional Generation: The Problem

Basic diffusion models: Generate images from random noise

- No control over what gets generated
- Could be anything: cat, dog, car, person...

What we want: Control the generation

- Generate only dogs
- Generate cats with specific features
- Generate images matching text: "a dog wearing sunglasses"

Random Noise → ???

No control!

Solution

We need **guidance** to steer the generation process toward our desired outcome

Interlude: What is a Score Function?

Recall DDPM: We learned to predict noise $\epsilon_\theta(x_t, t)$

Interlude: What is a Score Function?

Recall DDPM: We learned to predict noise $\epsilon_\theta(x_t, t)$

But there's another equivalent way to think about diffusion models...

Interlude: What is a Score Function?

Recall DDPM: We learned to predict noise $\epsilon_\theta(x_t, t)$

But there's another equivalent way to think about diffusion models...

Score Function Definition

The **score function** is the gradient of the log probability:

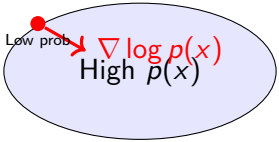
$$\text{score}(x_t) = \nabla_{x_t} \log p(x_t)$$

It tells us: *"Which direction increases the probability of x_t?"*

Interlude: What is a Score Function?

Intuition:

- $p(x_t)$ = how likely is this noisy image?
- $\nabla_{x_t} \log p(x_t)$ = which way to move to make it *more likely*?
- Points toward higher probability regions



Score Function vs Noise Prediction

Three equivalent formulations of diffusion models:

1. **Predict x_0:** Directly predict the clean image

Score Function vs Noise Prediction

Three equivalent formulations of diffusion models:

1. **Predict x_0:** Directly predict the clean image
2. **Predict noise ϵ:** What we've been doing in DDPM

Score Function vs Noise Prediction

Three equivalent formulations of diffusion models:

1. **Predict x_0:** Directly predict the clean image
2. **Predict noise ϵ:** What we've been doing in DDPM
3. **Predict score:** Learn $\nabla_{x_t} \log p(x_t)$

Score Function vs Noise Prediction

Three equivalent formulations of diffusion models:

1. **Predict x_0:** Directly predict the clean image
2. **Predict noise ϵ:** What we've been doing in DDPM
3. **Predict score:** Learn $\nabla_{x_t} \log p(x_t)$

Connection between Noise and Score

These are related by:

$$\nabla_{x_t} \log p(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

Score Function vs Noise Prediction

Three equivalent formulations of diffusion models:

1. **Predict x_0:** Directly predict the clean image
2. **Predict noise ϵ:** What we've been doing in DDPM
3. **Predict score:** Learn $\nabla_{x_t} \log p(x_t)$

Connection between Noise and Score

These are related by:

$$\nabla_{x_t} \log p(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

Why introduce score functions now?

- Classifier guidance is easier to understand with scores
- Score functions make the math cleaner
- They reveal the geometric intuition: "follow the gradient uphill"

Classifier Guidance

Adding conditions using a classifier

Main Idea: Use a pre-trained classifier to guide generation

Classifier Guidance

Adding conditions using a classifier

Main Idea: Use a pre-trained classifier to guide generation

How it works:

1. Train a regular diffusion model (unconditional)

Classifier Guidance

Adding conditions using a classifier

Main Idea: Use a pre-trained classifier to guide generation

How it works:

1. Train a regular diffusion model (unconditional)
2. Get a separate classifier (e.g., ImageNet classifier)

Classifier Guidance

Adding conditions using a classifier

Main Idea: Use a pre-trained classifier to guide generation

How it works:

1. Train a regular diffusion model (unconditional)
2. Get a separate classifier (e.g., ImageNet classifier)
3. At each denoising step: ask classifier "does this look like class y?"

Classifier Guidance

Adding conditions using a classifier

Main Idea: Use a pre-trained classifier to guide generation

How it works:

1. Train a regular diffusion model (unconditional)
2. Get a separate classifier (e.g., ImageNet classifier)
3. At each denoising step: ask classifier "does this look like class y?"
4. Push the image toward the desired class using gradients

Intuition

Think of the classifier as a "teacher" constantly saying:

- "That doesn't look enough like a dog yet!"
- "Move more in this direction to make it look like a dog"

Classifier Guidance: The Math

Unconditional score: $\nabla_{x_t} \log p(x_t)$

Classifier Guidance: The Math

Unconditional score: $\nabla_{x_t} \log p(x_t)$

Conditional score (what we want): $\nabla_{x_t} \log p(x_t|y)$

Classifier Guidance: The Math

Unconditional score: $\nabla_{x_t} \log p(x_t)$

Conditional score (what we want): $\nabla_{x_t} \log p(x_t|y)$

Using Bayes' theorem:

$$\begin{aligned}\nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log \frac{p(x_t)p(y|x_t)}{p(y)} \\ &= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)\end{aligned}$$

Classifier Guidance: The Math

Unconditional score: $\nabla_{x_t} \log p(x_t)$

Conditional score (what we want): $\nabla_{x_t} \log p(x_t|y)$

Using Bayes' theorem:

$$\begin{aligned}\nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log \frac{p(x_t)p(y|x_t)}{p(y)} \\ &= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)\end{aligned}$$

Classifier Guidance Formula

$$\boxed{\nabla_{x_t} \log p(x_t|y) = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional score}} + \underbrace{\nabla_{x_t} \log p(y|x_t)}_{\text{classifier gradient}}}$$

Classifier Guidance: The Math

Unconditional score: $\nabla_{x_t} \log p(x_t)$

Conditional score (what we want): $\nabla_{x_t} \log p(x_t|y)$

Using Bayes' theorem:

$$\begin{aligned}\nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log \frac{p(x_t)p(y|x_t)}{p(y)} \\ &= \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)\end{aligned}$$

Classifier Guidance Formula

$$\boxed{\nabla_{x_t} \log p(x_t|y) = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional score}} + \underbrace{\nabla_{x_t} \log p(y|x_t)}_{\text{classifier gradient}}}$$

Add guidance scale λ to control strength:

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \lambda \cdot \nabla_{x_t} \log p(y|x_t)$$

Classifier Guidance: Pros and Cons

Advantages

- Simple concept

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes
- Mathematically clean

Limitations

- Needs separate classifier

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes
- Mathematically clean

Limitations

- Needs separate classifier
- Classifier must work on noisy images

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes
- Mathematically clean

Limitations

- Needs separate classifier
- Classifier must work on noisy images
- Limited to classifier's classes

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes
- Mathematically clean

Limitations

- Needs separate classifier
- Classifier must work on noisy images
- Limited to classifier's classes
- Can't use text prompts

Classifier Guidance: Pros and Cons

Advantages

- Simple concept
- Works with existing models
- Strong control over classes
- Mathematically clean

Limitations

- Needs separate classifier
- Classifier must work on noisy images
- Limited to classifier's classes
- Can't use text prompts

Main Problem

If classifier has 10 classes, you can only generate those 10 classes!

- Want to generate class 11? Need to retrain classifier
- Want text-to-image? Classifiers don't use text

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y
- Unconditional generation: $p(x_t)$ without condition

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y
- Unconditional generation: $p(x_t)$ without condition

How to train both at once?

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y
- Unconditional generation: $p(x_t)$ without condition

How to train both at once?

During training, randomly drop the condition:

- 10% of time: use empty/null condition $\emptyset$
- 90% of time: use actual condition y

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y
- Unconditional generation: $p(x_t)$ without condition

How to train both at once?

During training, randomly drop the condition:

- 10% of time: use empty/null condition $\emptyset$
- 90% of time: use actual condition y

This way, the model learns both conditional and unconditional generation!

Classifier Free Guidance

Getting rid of the classifier!

Big Idea: What if we don't need a separate classifier?

Key Insight: Train one model that can do BOTH:

- Conditional generation: $p(x_t|y)$ with condition y
- Unconditional generation: $p(x_t)$ without condition

How to train both at once?

During training, randomly drop the condition:

- 10% of time: use empty/null condition $\emptyset$
- 90% of time: use actual condition y

This way, the model learns both conditional and unconditional generation!

Result: Single model, infinite flexibility, no external classifier needed!

Classifier-Free Guidance: The Math

Recall classifier guidance:

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \lambda \cdot \nabla_{x_t} \log p(y|x_t)$$

Classifier-Free Guidance: The Math

Recall classifier guidance:

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \lambda \cdot \nabla_{x_t} \log p(y|x_t)$$

Manipulate the gradient term:

$$\begin{aligned}\nabla_{x_t} \log p(y|x_t) &= \nabla_{x_t} \log \frac{p(x_t|y)}{p(x_t)} \\ &= \nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t)\end{aligned}$$

Classifier-Free Guidance: The Math

Recall classifier guidance:

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \lambda \cdot \nabla_{x_t} \log p(y|x_t)$$

Manipulate the gradient term:

$$\begin{aligned}\nabla_{x_t} \log p(y|x_t) &= \nabla_{x_t} \log \frac{p(x_t|y)}{p(x_t)} \\ &= \nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t)\end{aligned}$$

Substitute back:

$$\begin{aligned}\nabla_{x_t} \log p(x_t|y) &= \nabla_{x_t} \log p(x_t) + \lambda \cdot [\nabla_{x_t} \log p(x_t|y) - \nabla_{x_t} \log p(x_t)] \\ &= (1-\lambda)\nabla_{x_t} \log p(x_t) + \lambda \cdot \nabla_{x_t} \log p(x_t|y)\end{aligned}$$

Classifier-Free Guidance: The Final Loss formulation

Classifier-Free Guidance Formula

$$\boxed{\nabla_{x_t} \log p(x_t|y) = \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional}} + \lambda \cdot [\underbrace{\nabla_{x_t} \log p(x_t|y)}_{\text{conditional}} - \underbrace{\nabla_{x_t} \log p(x_t)}_{\text{unconditional}}]}$$

Understanding Classifier-Free Guidance

The Formula (in noise space)

$$\boxed{\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})}$$

Understanding Classifier-Free Guidance

The Formula (in noise space)

$$\boxed{\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})}$$

Think of it as a vector:

- Find the direction from unconditional to conditional

Understanding Classifier-Free Guidance

The Formula (in noise space)

$$\boxed{\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})}$$

Think of it as a vector:

- Find the direction from unconditional to conditional
- Amplify this direction by λ (guidance scale)

Understanding Classifier-Free Guidance

The Formula (in noise space)

$$\boxed{\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})}$$

Think of it as a vector:

- Find the direction from unconditional to conditional
- Amplify this direction by λ (guidance scale)
- Higher λ -> stronger adherence to condition

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything
   - Simpler architecture

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything
   - Simpler architecture
2. **Unlimited conditions**
   - Not limited to fixed classes
   - Works with text, images, any modality

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything
   - Simpler architecture
2. **Unlimited conditions**
   - Not limited to fixed classes
   - Works with text, images, any modality
3. **Better quality**
   - More control over generation
   - Used in DALLÂ · E 2, Imagen, Stable Diffusion

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything
   - Simpler architecture
2. **Unlimited conditions**
   - Not limited to fixed classes
   - Works with text, images, any modality
3. **Better quality**
   - More control over generation
   - Used in DALLÂ · E 2, Imagen, Stable Diffusion
4. **Flexible**
   - Adjust guidance scale at inference time
   - No retraining needed

Classifier-Free Guidance: Advantages

Why is it so popular?

1. **No external classifier needed**
   - Single model does everything
   - Simpler architecture
2. **Unlimited conditions**
   - Not limited to fixed classes
   - Works with text, images, any modality
3. **Better quality**
   - More control over generation
   - Used in DALLÂ · E 2, Imagen, Stable Diffusion
4. **Flexible**
   - Adjust guidance scale at inference time
   - No retraining needed

Current Standard

Classifier-free guidance is the **dominant approach** in modern text-to-image models!

Comparison: Classifier vs Classifier-Free

Aspect	Classifier Guidance	Classifier-Free Guidance
External Model	Needs separate classifier	Single model
Training	Train diffusion model only	Train diffusion model with dropout
Condition Types	Limited to classifier classes	Any condition (text, image, etc.)
Flexibility	Fixed classes	Infinite possibilities
Noisy Images	Classifier must handle noise	No issue
Scalability	Need to retrain classifier	Just train diffusion model
Computational Cost	1 x forward + classifier	2 x forward passes
Quality	Good	Better (typically)
Usage Today	Less common	Standard approach

Winner

Classifier-Free Guidance is preferred in practice due to flexibility and quality

How Classifier-Free Guidance is Trained

Training procedure:

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)
2. During training, randomly drop the condition:
   - With probability 10%: replace c with null condition $\emptyset$
   - With probability 90%: keep c

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)
2. During training, randomly drop the condition:
   - With probability 10%: replace c with null condition $\emptyset$
   - With probability 90%: keep c
3. Train model to predict noise: $\epsilon_\theta(x_t, t, c)$

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)
2. During training, randomly drop the condition:
   - With probability 10%: replace c with null condition $\emptyset$
   - With probability 90%: keep c
3. Train model to predict noise: $\epsilon_\theta(x_t, t, c)$
4. The conditioning c is converted to a vector and added to the UNet model present internally(just like time embedding in DDPM)

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)
2. During training, randomly drop the condition:
   - With probability 10%: replace c with null condition $\emptyset$
   - With probability 90%: keep c
3. Train model to predict noise: $\epsilon_\theta(x_t, t, c)$
4. The conditioning c is converted to a vector and added to the UNet model present internally(just like time embedding in DDPM)
5. Model learns BOTH:
   - Conditional: when c is provided
   - Unconditional: when $c = \emptyset$

How Classifier-Free Guidance is Trained

Training procedure:

1. Take a dataset with pairs: (x, c) where c is condition (text, class, etc.)
2. During training, randomly drop the condition:
   - With probability 10%: replace c with null condition $\emptyset$
   - With probability 90%: keep c
3. Train model to predict noise: $\epsilon_\theta(x_t, t, c)$
4. The conditioning c is converted to a vector and added to the UNet model present internally(just like time embedding in DDPM)
5. Model learns BOTH:
   - Conditional: when c is provided
   - Unconditional: when $c = \emptyset$

Result

At inference time, you can:

- Get ϵ_{cond} by passing condition, Get ϵ_{uncond} by passing null condition.
- Combine them for guided generation!

Summary: Guidance in Diffusion Models

Classifier Guidance

$$\epsilon = \epsilon_{\text{uncond}} + \lambda \cdot \underbrace{\nabla_{x_t} \log p(y|x_t)}_{\text{classifier gradient}}$$

Pros: Simple, works with existing models
Cons: Needs classifier, limited to fixed classes

Classifier-Free Guidance (CFG)

$$\epsilon = \epsilon_{\text{uncond}} + \lambda \cdot (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

Pros: No classifier, unlimited conditions, better quality
Cons: Requires training with condition dropout

The Latent Diffusion models

Emergence of Stable Diffusion Architecture

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware
4. **Slow generation**: Minutes per image

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware
4. **Slow generation**: Minutes per image

Key Observation:

- Not all pixels are equally important

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware
4. **Slow generation**: Minutes per image

Key Observation:

- Not all pixels are equally important
- Many pixels are redundant (e.g., clear blue sky)

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware
4. **Slow generation**: Minutes per image

Key Observation:

- Not all pixels are equally important
- Many pixels are redundant (e.g., clear blue sky)
- High-frequency details matter less for overall structure

The Problem with Pixel Space Diffusion

DDPM works directly on images (pixels)

Example: 512 x 512 RGB image = 512 x 512 x 3 = 786,432 dimensions!

Major Issues

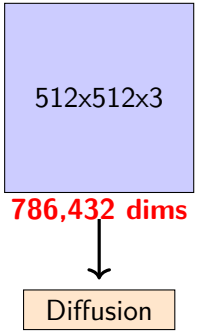
1. **Computational cost**: Every denoising step processes 786k values
2. **Memory intensive**: Need powerful GPUs
3. **Slow training**: Days/weeks on expensive hardware
4. **Slow generation**: Minutes per image

Key Observation:

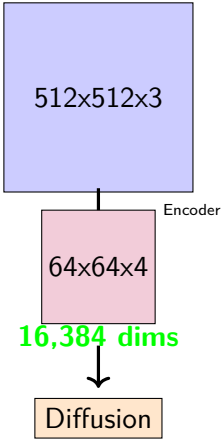
- Not all pixels are equally important
- Many pixels are redundant (e.g., clear blue sky)
- High-frequency details matter less for overall structure

Pixel Space vs Latent Space

Pixel Space Diffusion



Latent Space Diffusion



Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Latent Space Diffusion Characteristics:

- Low dimensional

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Latent Space Diffusion Characteristics:

- Low dimensional
- Fast (48x less!)

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Latent Space Diffusion Characteristics:

- Low dimensional
- Fast (48x less!)
- Efficient

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Latent Space Diffusion Characteristics:

- Low dimensional
- Fast (48x less!)
- Efficient
- Better quality

Pixel Space vs Latent Space

Pixel Space Diffusion Characteristics:

- High dimensional
- Slow
- Expensive
- Direct control

Latent Space Diffusion Characteristics:

- Low dimensional
- Fast (48x less!)
- Efficient
- Better quality

Key Insight

Compress image to compact latent representation -> run diffusion there -> decompress!

What is Latent Space?

Latent space = compressed, meaningful representation of data

What is Latent Space?

Latent space = compressed, meaningful representation of data

Analogy: Think of it like JPEG compression

- Original image: large, many pixels

What is Latent Space?

Latent space = compressed, meaningful representation of data

Analogy: Think of it like JPEG compression

- Original image: large, many pixels
- JPEG: smaller, compressed, still recognizable

What is Latent Space?

Latent space = compressed, meaningful representation of data

Analogy: Think of it like JPEG compression

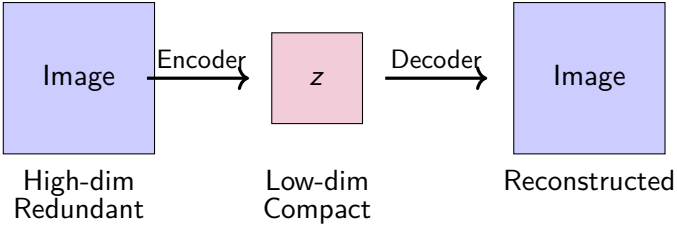
- Original image: large, many pixels
- JPEG: smaller, compressed, still recognizable
- Latent space: even more compressed, captures "essence"

What is Latent Space?

Latent space = compressed, meaningful representation of data

Analogy: Think of it like JPEG compression

- Original image: large, many pixels
- JPEG: smaller, compressed, still recognizable
- Latent space: even more compressed, captures "essence"



The Autoencoder: Creating Latent Space

How to build a good latent space? -> Use an **Autoencoder**

The Autoencoder: Creating Latent Space

How to build a good latent space? -> Use an **Autoencoder**

Autoencoder Components

1. **Encoder** $\mathcal{E}$: Image $x \rightarrow$ Latent $z = \mathcal{E}(x)$

The Autoencoder: Creating Latent Space

How to build a good latent space? -> Use an **Autoencoder**

Autoencoder Components

1. **Encoder** $\mathcal{E}$: Image $x \rightarrow$ Latent $z = \mathcal{E}(x)$
2. **Decoder** $\mathcal{D}$: Latent $z \rightarrow$ Image $\hat{x} = \mathcal{D}(z)$

The Autoencoder: Creating Latent Space

How to build a good latent space? -> Use an **Autoencoder**

Autoencoder Components

1. **Encoder** $\mathcal{E}$: Image $x \rightarrow$ Latent $z = \mathcal{E}(x)$
2. **Decoder** $\mathcal{D}$: Latent $z \rightarrow$ Image $\hat{x} = \mathcal{D}(z)$

Goal: $\hat{x} \approx x$ (reconstruction)

The Autoencoder: Creating Latent Space

How to build a good latent space? -> Use an **Autoencoder**

Autoencoder Components

1. **Encoder** $\mathcal{E}$: Image $x \to$ Latent $z = \mathcal{E}(x)$
2. **Decoder** $\mathcal{D}$: Latent $z \to$ Image $\hat{x} = \mathcal{D}(z)$

Goal: $\hat{x} \approx x$ (reconstruction)

Training objective:

$$\mathcal{L} = \|x - \mathcal{D}(\mathcal{E}(x))\|^2 + \text{regularization}$$

Latent Diffusion Models: The Complete Picture

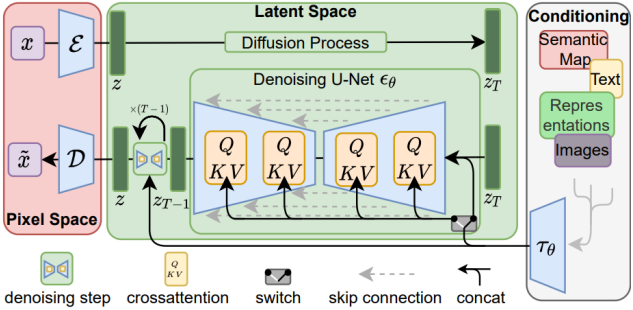


Figure: Latent DIffusion Model Architecture

Latent Diffusion Models: Training

Two-stage training process:

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents
4. Once trained: **freeze** (don't update anymore)

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents
4. Once trained: **freeze** (don't update anymore)

Stage 2: Train the Diffusion Model

1. Encode training images: $z = \mathcal{E}(x)$

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents
4. Once trained: **freeze** (don't update anymore)

Stage 2: Train the Diffusion Model

1. Encode training images: $z = \mathcal{E}(x)$
2. Add noise to latents: $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents
4. Once trained: **freeze** (don't update anymore)

Stage 2: Train the Diffusion Model

1. Encode training images: $z = \mathcal{E}(x)$
2. Add noise to latents: $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$
3. Train U-Net to predict noise: $\mathcal{L} = \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$

Latent Diffusion Models: Training

Two-stage training process:

Stage 1: Train the Autoencoder (VAE)

1. Collect large dataset of images
2. Train encoder $\mathcal{E}$ and decoder $\mathcal{D}$
3. Goal: $\mathcal{D}(\mathcal{E}(x)) \approx x$ with small latents
4. Once trained: **freeze** (don't update anymore)

Stage 2: Train the Diffusion Model

1. Encode training images: $z = \mathcal{E}(x)$
2. Add noise to latents: $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$
3. Train U-Net to predict noise: $\mathcal{L} = \|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$
4. Condition c can be text embeddings, class labels, etc.

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$
   - Size: e.g., 64x64x4 (not 512x512x3!)

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$
   - Size: e.g., 64x64x4 (not 512x512x3!)
2. **Provide condition:** Text prompt c (e.g., "a cat on a sofa")

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$
   - Size: e.g., 64x64x4 (not 512x512x3!)
2. **Provide condition:** Text prompt c (e.g., "a cat on a sofa")
3. **Iterative denoising in latent space:**
   - For $t = T, T-1, ..., 1$:
   - Predict noise: $\epsilon_\theta(z_t, t, c)$
   - Remove noise: $z_{t-1} = \text{denoise}(z_t, \epsilon_\theta)$

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$
   - Size: e.g., 64x64x4 (not 512x512x3!)
2. **Provide condition:** Text prompt c (e.g., "a cat on a sofa")
3. **Iterative denoising in latent space:**
   - For $t = T, T-1, ..., 1$:
   - Predict noise: $\epsilon_\theta(z_t, t, c)$
   - Remove noise: $z_{t-1} = \text{denoise}(z_t, \epsilon_\theta)$
4. **Decode to pixel space:** $x = \mathcal{D}(z_0)$

Latent Diffusion Models: Generation

How to generate an image:

1. **Start with random latent noise:** $z_T \sim \mathcal{N}(0, I)$
   - Size: e.g., 64x64x4 (not 512x512x3!)
2. **Provide condition:** Text prompt c (e.g., "a cat on a sofa")
3. **Iterative denoising in latent space:**
   - For $t = T, T-1, ..., 1$:
   - Predict noise: $\epsilon_\theta(z_t, t, c)$
   - Remove noise: $z_{t-1} = \text{denoise}(z_t, \epsilon_\theta)$
4. **Decode to pixel space:** $x = \mathcal{D}(z_0)$

Speed-up Achieved

- Latent space: 64x64x4 = 16,384 dims
- Pixel space: 512x512x3 = 786,432 dims
- **48x reduction** in dimensions -> much faster!

Cross-Attention: How Text Controls Generation

How does text prompt influence the image? -> Cross-Attention

Cross-Attention: How Text Controls Generation

How does text prompt influence the image? -> Cross-Attention

Cross-Attention Mechanism

At each U-Net layer:

$$Q = W_Q \cdot z_t \quad \text{(Query: from image latent)}$$
$$K = W_K \cdot c \quad \text{(Key: from text embedding)}$$
$$V = W_V \cdot c \quad \text{(Value: from text embedding)}$$
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

Cross-Attention: How Text Controls Generation

How does text prompt influence the image? -> Cross-Attention

Cross-Attention Mechanism

At each U-Net layer:

$$
\begin{aligned}
Q &= W_Q \cdot z_t \quad \text{(Query: from image latent)} \\
K &= W_K \cdot c \quad \text{(Key: from text embedding)} \\
V &= W_V \cdot c \quad \text{(Value: from text embedding)}
\end{aligned}
$$

$$
\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V
$$

Intuition:

- Image asks: "What features should I have here?"

Cross-Attention: How Text Controls Generation

How does text prompt influence the image? -> Cross-Attention

Cross-Attention Mechanism

At each U-Net layer:

$$Q = W_Q \cdot z_t \quad \text{(Query: from image latent)}$$
$$K = W_K \cdot c \quad \text{(Key: from text embedding)}$$
$$V = W_V \cdot c \quad \text{(Value: from text embedding)}$$
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

Intuition:

- Image asks: "What features should I have here?"
- Text provides: "A cat should have whiskers, fur texture..."

Cross-Attention: How Text Controls Generation

How does text prompt influence the image? -> Cross-Attention

Cross-Attention Mechanism

At each U-Net layer:

$$
\begin{aligned}
Q &= W_Q \cdot z_t \quad \text{(Query: from image latent)} \\
K &= W_K \cdot c \quad \text{(Key: from text embedding)} \\
V &= W_V \cdot c \quad \text{(Value: from text embedding)}
\end{aligned}
$$

$$
\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V
$$

Intuition:

- Image asks: "What features should I have here?"
- Text provides: "A cat should have whiskers, fur texture..."
- Attention decides which text features to use where

Stable Diffusion: LDM in Practice

Stable Diffusion = Latent Diffusion Model released by Stability AI

Stable Diffusion: LDM in Practice

Stable Diffusion = Latent Diffusion Model released by Stability AI

Key Specifications

- **VAE compression factor:** $f = 8$ ($512 \times 512 \rightarrow 64 \times 64$)
- **Latent channels:** 4
- **Text encoder:** CLIP ViT-L/14 (for SD 1.x/2.x)
- **U-Net:** Cross-attention at multiple resolutions
- **Training data:** LAION-5B (billions of text-image pairs)

Stable Diffusion: LDM in Practice

Stable Diffusion = Latent Diffusion Model released by Stability AI

Key Specifications

- **VAE compression factor:** $f = 8$ ($512 \times 512 \rightarrow 64 \times 64$)
- **Latent channels:** 4
- **Text encoder:** CLIP ViT-L/14 (for SD 1.x/2.x)
- **U-Net:** Cross-attention at multiple resolutions
- **Training data:** LAION-5B (billions of text-image pairs)

Impact

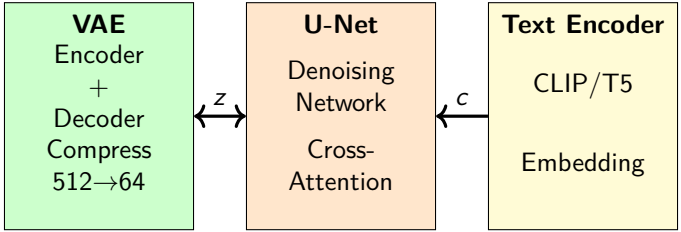
First high-quality, **open-source** text-to-image model that can run on consumer GPUs (RTX 3090, even RTX 3060)

Stable Diffusion Versions

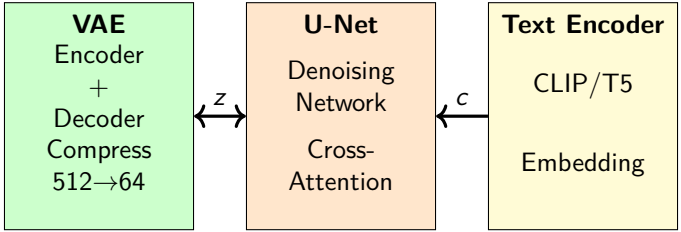
Version	Release	Key Features
SD 1.4	Aug 2022	First public release, 512x512
SD 1.5	Oct 2022	Improved quality, same architecture
SD 2.0	Nov 2022	New text encoder (OpenCLIP) 512x512 and 768x768 variants
SD 2.1	Dec 2022	Better text understanding
SDXL 1.0	Jul 2023	**1024x1024**, dual text encoders Refiner model for quality
SD 3.0	2024	Multimodal Diffusion Transformer (DiT architecture)

All based on Latent Diffusion Models (LDM) architecture!

LDM Architecture Summary



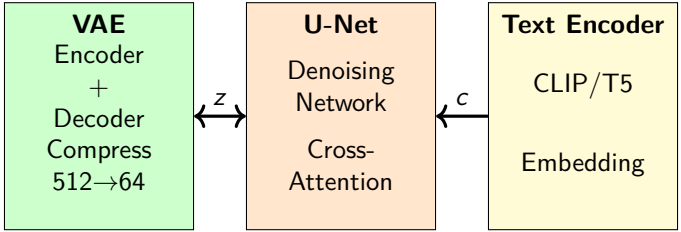
LDM Architecture Summary



Complete Pipeline

$$\text{Text} \xrightarrow{\text{CLIP}} c \rightarrow \text{U-Net}(z_t, t, c) \rightarrow z_0 \xrightarrow{\text{Decoder}} \text{Image}$$

LDM Architecture Summary

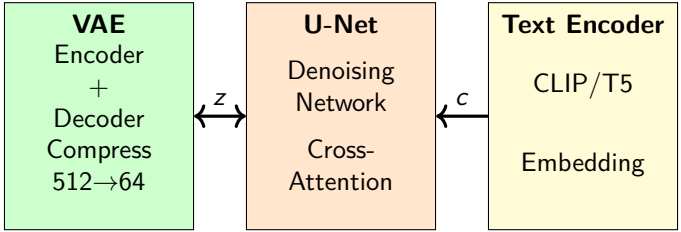


Complete Pipeline

$$\text{Text} \xrightarrow{\text{CLIP}} c \rightarrow \text{U-Net}(z_t, t, c) \rightarrow z_0 \xrightarrow{\text{Decoder}} \text{Image}$$

Training: Minimize $\mathbb{E}_{z,\epsilon,t,c}\left[\|\epsilon - \epsilon_\theta(z_t, t, c)\|^2\right]$

LDM Architecture Summary



Complete Pipeline

$$\text{Text} \xrightarrow{\text{CLIP}} c \rightarrow \text{U-Net}(z_t, t, c) \rightarrow z_0 \xrightarrow{\text{Decoder}} \text{Image}$$

Training: Minimize $\mathbb{E}_{z,\epsilon,t,c}\left[\|\epsilon - \epsilon_\theta(z_t, t, c)\|^2\right]$

Inference: Sample z_T, denoise with conditioning, decode back to pixel space.

Conclusion

From Thermodynamics to Art

1. The Paradigm Shift

- We moved from "One Hard Step" (GANs) to **"Many Easy Steps"** (Diffusion).

Conclusion

From Thermodynamics to Art

1. The Paradigm Shift

- We moved from "One Hard Step" (GANs) to **"Many Easy Steps"** (Diffusion).
- **The Trade-off:** We gained training stability and mode coverage at the cost of inference speed.

2. Evolution of the Architecture

- **Speed (DDPM $\rightarrow$ DDIM):** Deterministic sampling reduced inference from 1000 steps to $\approx$ 50 steps.

Conclusion

From Thermodynamics to Art

1. The Paradigm Shift

- We moved from "One Hard Step" (GANs) to **"Many Easy Steps"** (Diffusion).
- **The Trade-off:** We gained training stability and mode coverage at the cost of inference speed.

2. Evolution of the Architecture

- **Speed (DDPM $\rightarrow$ DDIM):** Deterministic sampling reduced inference from 1000 steps to $\approx$ 50 steps.
- **Scale (Pixel $\rightarrow$ Latent):** Perceptual compression (LDM) moved the heavy lifting to a lower-dimensional space.

Conclusion

From Thermodynamics to Art

1. The Paradigm Shift

- We moved from "One Hard Step" (GANs) to **"Many Easy Steps"** (Diffusion).
- **The Trade-off:** We gained training stability and mode coverage at the cost of inference speed.

2. Evolution of the Architecture

- **Speed (DDPM $\rightarrow$ DDIM):** Deterministic sampling reduced inference from 1000 steps to $\approx$ 50 steps.
- **Scale (Pixel $\rightarrow$ Latent):** Perceptual compression (LDM) moved the heavy lifting to a lower-dimensional space.

The Fundamental Insight

Diffusion models succeed because they simplify the generation problem:

Don't learn to create an image from scratch; just learn to fix a slightly noisy one.